

JOBSHEET XII
Double Linked List



Disusun oleh:

Nama : Galih Candra Kirana
No. Absen : 10
Kelas : TI 1G
NIM : 254107020080

POLITEKNIK NEGERI MALANG
Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141
TAHUN 2025-2026

12.1 Tujuan Praktikum

Tujuan Praktikum Setelah melakukan praktikum ini, mahasiswa mampu:

1. Memahami algoritma double linked list;
2. Membuat dan mendeklarasikan struktur algoritma double linked list;
3. Menerapkan operasi penambahan, pencarian, dan penghapusan data pada Double Linked List.

12.2 Percobaan 1: Operasi Penambahan pada Double Linked List

12.2.1 Langkah-langkah Percobaan

Class Mahasiswa10:

```
package Jobsheet12;

public class Mahasiswa10 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa10(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampil(){
        System.out.println(
            "NIM      : " + nim +
            "\nNama    : " + nama +
            "\nKelas   : " + kelas +
            "\nIPK     : " + ipk
        );
    }
}
```

Class Node10:

```
package Jobsheet12;

public class Node10 {
    Mahasiswa10 data;
    Node10 prev;
    Node10 next;

    public Node10(Mahasiswa10 data) {
        this.data = data;
        this.prev = null;
    }
}
```

```

        this.next = null;
    }
}

```

Class DoubleLinkedList10:

```

package Jobsheet12;

public class DoubleLinkedList10 {
    Node10 head;
    Node10 tail;

    public DoubleLinkedList10() {
        head = null;
        tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void addFirst(Mahasiswa10 data) {
        Node10 newNode = new Node10(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    public void addLast(Mahasiswa10 data) {
        Node10 newNode = new Node10(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    public void insertAfter(String keyNim, Mahasiswa10 data) {
        Node10 current = head;
        while (current != null && !current.data.nim.equals(keyNim)) {
            current = current.next;
        }
    }
}

```

```

        if (current == null) {
            System.out.println("Data dengan NIM " + keyNim + " tidak
ditemukan.");
            return;
        }

        Node10 newNode = new Node10(data);

        if (current == tail) {
            newNode.prev = current;
            current.next = newNode;
            tail = newNode;
        } else {
            newNode.prev = current;
            newNode.next = current.next;
            current.next.prev = newNode;
            current.next = newNode;
        }
        System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
    }

    public void print() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong");
            return;
        }

        Node10 current = head;
        while (current != null) {
            current.data.tampil();
            current = current.next;
        }
    }
}

```

Class DoubleLinkedListMain10:

```

package Jobsheet12;

import java.util.Scanner;

public class DoubleLinkedListMain10 {

    static public Mahasiswa10 inputMahasiswa(Scanner sc) {
        System.out.print("Masukkan NIM   : ");
        String nim = sc.nextLine();
        System.out.print("Masukkan nama  : ");
        String nama = sc.nextLine();
        System.out.print("Masukkan kelas : ");
    }
}

```

```

String kelas = sc.nextLine();
System.out.print("Masukkan ipk   : ");
double ipk = sc.nextDouble();
Mahasiswa10 data = new Mahasiswa10(nim, nama, kelas, ipk);
return data;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    DoubleLinkedList10 list = new DoubleLinkedList10();
    int pil = 0;

    do {
        System.out.println("\n ===== MENU DOUBLE LINKED LIST");
        System.out.println("1. Tambah data di awal");
        System.out.println("2. Tambah data di akhir");
        System.out.println("3. Sisipkan data di tengah (setelah NIM)");
        System.out.println("4. Hapus data di awal");
        System.out.println("5. Hapus data di akhir");
        System.out.println("6. Tampilkan data");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu: ");
        pil = sc.nextInt();
        sc.nextLine();

        switch (pil) {
            case 1:
                Mahasiswa10 mhsAwal = inputMahasiswa(sc);
                list.addFirst(mhsAwal);
                break;

            case 2:
                Mahasiswa10 mhsAkhir = inputMahasiswa(sc);
                list.addLast(mhsAkhir);
                break;

            case 3:
                System.out.print("Masukkan NIM yang dicari: ");
                String keyNim = sc.nextLine();
                System.out.println("Masukkan data baru: ");
                Mahasiswa10 dataBaru = inputMahasiswa(sc);
                list.insertAfter(keyNim, dataBaru);
                break;

            case 4:
                // list.removeFirst();
                break;

            case 5:
                // list.removeLast();

```

```

        break;

    case 6:
        list.print();
        break;

    case 0:
        System.out.println("Keluar.");
        break;

    default:
        System.out.println("Pilihan tidak valid.");
        break;
    }
} while (pil != 0);

sc.close();
}
}

```

12.2.2 Verifikasi Hasil Percobaan

```

===== MENU DOUBLE LINKED LIST
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 2
Masukkan NIM      : 123005
Masukkan nama     : Harry
Masukkan kelas    : 1A
Masukkan ipk      : 3,76

===== MENU DOUBLE LINKED LIST
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 3
Masukkan NIM yang dicari: 123005
Masukkan data baru:
Masukkan NIM      : 123010
Masukkan nama     : Potter
Masukkan kelas    : 18
Masukkan ipk      : 3,55
Data berhasil disisipkan setelah NIM 123005

```

```

===== MENU DOUBLE LINKED LIST
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 6
NIM      : 123005
Nama     : Harry
Kelas   : 1A
IPK      : 3.76
NIM      : 123010
Nama     : Potter
Kelas   : 18
IPK      : 3.55

```

12.2.3 Pertanyaan!

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!

Jawaban:

Perbedaannya pada single linked list hanya memiliki satu pointer yaitu next, sedangkan pada double linked list, ada dua pointer yaitu next dan previous.

2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!

Jawaban:

Fungsi atribut next digunakan untuk menyimpan alamat node berpindah berikutnya, sehingga proses traversal dapat menuju ke node berikutnya. Sedangkan fungsi atribut prev digunakan untuk menyimpan alamat node sebelumnya, sehingga proses traversal dapat menuju ke node sebelumnya.

3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!

Jawaban:

Fungsi konstruktor tersebut adalah untuk menginisialisasi kondisi awal linked list. Ketika linked list dibuat, head dan tail diatur null karena belum ada node di dalamnya.

4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode;  
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

Jawaban:

Karena pada saat node masih kosong dan akan ditambahkan 1 node baru, maka node tersebut satu-satunya node di dalam linked list. Oleh sebab itu, head dan tail harus sama-sama menunjuk ke node tersebut.

5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!

Jawaban:

```
public void print() {  
    if (isEmpty()) {  
        System.out.println("Linked List masih kosong");  
        return;  
    }  
  
    Node10 current = head;  
    while (current != null) {  
        current.data.tampil();  
        current = current.next;  
    }  
}
```

6. Modifikasi kode program dengan menambahkan method `printReverse()` untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

Jawaban:

```
public void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong");
        return;
    }

    Node10 current = tail;
    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}
```

12.3 Percobaan 2: Operasi Penghapusan pada Double Linked List

12.3.1 Langkah – langkah percobaan

Class DoubleLinkedList10:

```
package Jobsheet12;

public class DoubleLinkedList10 {
    Node10 head;
    Node10 tail;

    public DoubleLinkedList10() {
        head = null;
        tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void addFirst(Mahasiswa10 data) {
        Node10 newNode = new Node10(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }
}
```

```

    }

    public void addLast(Mahasiswa10 data) {
        Node10 newNode = new Node10(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    public void insertAfter(String keyNim, Mahasiswa10 data) {
        Node10 current = head;
        while (current != null && !current.data.nim.equals(keyNim)) {
            current = current.next;
        }
        if (current == null) {
            System.out.println("Data dengan NIM " + keyNim + " tidak
ditemukan.");
            return;
        }

        Node10 newNode = new Node10(data);

        if (current == tail) {
            newNode.prev = current;
            current.next = newNode;
            tail = newNode;
        } else {
            newNode.prev = current;
            newNode.next = current.next;
            current.next.prev = newNode;
            current.next = newNode;
        }
        System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
    }

    public void print() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong");
            return;
        }

        Node10 current = head;
        while (current != null) {
            current.data.tampil();
            current = current.next;
        }
    }

```

```

    }

    public void printReverse() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong");
            return;
        }

        Node10 current = tail;
        while (current != null) {
            current.data.tampil();
            current = current.prev;
        }
    }

    public void removeFirst() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong.");
        }

        if (head == tail) {
            head = tail = null;
            System.out.println("Data berhasil dihapus.");
        } else {
            head = head.next;
            head.prev = null;
            System.out.println("Data berhasil dihapus.");
        }
    }

    public void removeLast() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong.");
        }

        if (head == tail) {
            head = tail = null;
            System.out.println("Data berhasil dihapus.");
        } else {
            tail = tail.prev;
            tail.next = null;
            System.out.println("Data berhasil dihapus.");
        }
    }
}

```

Class DoubleLinkedListMain10:

```
package Jobsheet12;
```

```

import java.util.Scanner;

public class DoubleLinkedListMain10 {

    static public Mahasiswa10 inputMahasiswa(Scanner sc) {
        System.out.print("Masukkan NIM   : ");
        String nim = sc.nextLine();
        System.out.print("Masukkan nama  : ");
        String nama = sc.nextLine();
        System.out.print("Masukkan kelas : ");
        String kelas = sc.nextLine();
        System.out.print("Masukkan ipk   : ");
        double ipk = sc.nextDouble();
        Mahasiswa10 data = new Mahasiswa10(nim, nama, kelas, ipk);
        return data;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        DoubleLinkedList10 list = new DoubleLinkedList10();
        int pil = 0;

        do {
            System.out.println("\n ===== MENU DOUBLE LINKED LIST");
            System.out.println("1. Tambah data di awal");
            System.out.println("2. Tambah data di akhir");
            System.out.println("3. Sisipkan data di tengah (setelah NIM)");
            System.out.println("4. Hapus data di awal");
            System.out.println("5. Hapus data di akhir");
            System.out.println("6. Tampilkan data");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            pil = sc.nextInt();
            sc.nextLine();

            switch (pil) {
                case 1:
                    Mahasiswa10 mhsAwal = inputMahasiswa(sc);
                    list.addFirst(mhsAwal);
                    break;

                case 2:
                    Mahasiswa10 mhsAkhir = inputMahasiswa(sc);
                    list.addLast(mhsAkhir);
                    break;

                case 3:
                    System.out.print("Masukkan NIM yang dicari: ");
                    String keyNim = sc.nextLine();
                    System.out.println("Masukkan data baru: ");

```

```
        Mahasiswa10 dataBaru = inputMahasiswa(sc);
        list.insertAfter(keyNim, dataBaru);
        break;

    case 4:
        list.removeFirst();
        break;

    case 5:
        list.removeLast();
        break;

    case 6:
        list.print();
        break;

    case 0:
        System.out.println("Keluar.");
        break;

    default:
        System.out.println("Pilihan tidak valid.");
        break;
    }
} while (pil != 0);

sc.close();
}
```

12.3.2 Verifikasi Hasil Percobaan

```
===== MENU DOUBLE LINKED LIST
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 5
NIM      : 123010
Nama     : Potter
Kelas   : 1B
IPK      : 3.55
Data berhasil dihapus.

===== MENU DOUBLE LINKED LIST
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 4
NIM      : 123005
Nama     : Harry
Kelas   : 1A
IPK      : 3.76
Data berhasil dihapus.
```

12.3.3 Pertanyaan!

1. Perhatikan potongan kode berikut pada method removeFirst()

```
head = head.next;
head.prev = null;
```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

Jawaban:

Fungsi statement yang pertama adalah mengganti head baru dengan head.next awal. Sedangkan statement yang kedua adalah menginisialisasi head.prev yang baru dengan nilai null.

2. Modifikasi method removeFirst() dan removeLast() agar program menampilkan data yang berhasil dihapus

Jawaban:

Class DoubleLinkedListMain10:

```
package Jobsheet12;

import java.util.Scanner;
```

```

public class DoubleLinkedListMain10 {

    static public Mahasiswa10 inputMahasiswa(Scanner sc) {
        System.out.print("Masukkan NIM   : ");
        String nim = sc.nextLine();
        System.out.print("Masukkan nama   : ");
        String nama = sc.nextLine();
        System.out.print("Masukkan kelas : ");
        String kelas = sc.nextLine();
        System.out.print("Masukkan ipk   : ");
        double ipk = sc.nextDouble();
        Mahasiswa10 data = new Mahasiswa10(nim, nama, kelas, ipk);
        return data;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        DoubleLinkedList10 list = new DoubleLinkedList10();
        int pil = 0;

        do {
            System.out.println("\n ===== MENU DOUBLE LINKED LIST");
            System.out.println("1. Tambah data di awal");
            System.out.println("2. Tambah data di akhir");
            System.out.println("3. Sisipkan data di tengah (setelah NIM)");
            System.out.println("4. Hapus data di awal");
            System.out.println("5. Hapus data di akhir");
            System.out.println("6. Tampilkan data");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            pil = sc.nextInt();
            sc.nextLine();

            switch (pil) {
                case 1:
                    Mahasiswa10 mhsAwal = inputMahasiswa(sc);
                    list.addFirst(mhsAwal);
                    break;

                case 2:
                    Mahasiswa10 mhsAkhir = inputMahasiswa(sc);
                    list.addLast(mhsAkhir);
                    break;

                case 3:
                    System.out.print("Masukkan NIM yang dicari: ");
                    String keyNim = sc.nextLine();
                    System.out.print("Masukkan data baru: ");
                    Mahasiswa10 dataBaru = inputMahasiswa(sc);
                    list.insertAfter(keyNim, dataBaru);
            }
        } while (pil != 0);
    }
}

```

```
        break;

    case 4:
        list.head.data.tampil();
        list.removeFirst();
        break;

    case 5:
        list.tail.data.tampil();
        list.removeLast();
        break;

    case 6:
        list.print();
        break;

    case 0:
        System.out.println("Keluar.");
        break;

    default:
        System.out.println("Pilihan tidak valid.");
        break;
    }
} while (pil != 0);

sc.close();
}
```